

PEDRO COSTA

**SENTINEL RISK SERVICE:
IDEMPOTÊNCIA, OUTBOX TRANSACIONAL E DECISÕES ANTIFRAUDE
EXPLICÁVEIS**

SÃO PAULO

2026

PEDRO COSTA

**SENTINEL RISK SERVICE:
IDEMPOTÊNCIA, OUTBOX TRANSACIONAL E DECISÕES ANTIFRAUDE
EXPLICÁVEIS**

Prova de conceito técnica apresentada como parte da série Projetos 2026 DEV, com o objetivo de documentar decisões de arquitetura, consistência, segurança e qualidade adotadas no desenvolvimento do Sentinel Risk Service.

SÃO PAULO

2026

RESUMO

Esta prova de conceito descreve o Sentinel Risk Service, um microsserviço em Java 21 e Spring Boot 4.1 criado para estudar análise antifraude explicável, idempotência e publicação confiável de eventos. A API recebe dados de uma transação, consulta a velocidade recente do cliente e executa cinco regras determinísticas. O resultado contém pontuação, decisão e fatores. A chave de idempotência é combinada com um fingerprint SHA-256 do comando, o que permite repetir a mesma solicitação sem criar outra análise e impede o reaproveitamento da chave com conteúdo diferente. A análise e o evento de outbox são gravados na mesma transação PostgreSQL. Um relay publica eventos pendentes no Kafka e mantém falhas disponíveis para nova tentativa. A solução assume entrega pelo menos uma vez e exige deduplicação do consumidor pelo identificador do evento. A avaliação reuniu testes unitários e de integração, cobertura mínima de 85%, Checkstyle, SpotBugs, CodeQL e SBOM CycloneDX. As regras são didáticas e não devem autorizar transações financeiras reais.

Palavras-chave: Microsserviços. Idempotência. Outbox transacional. Kafka. Antifraude.

ABSTRACT

This proof of concept describes Sentinel Risk Service, a Java 21 and Spring Boot 4.1 microservice built to study explainable fraud analysis, idempotency, and reliable event publication. The API evaluates five deterministic rules and returns the score, decision, and contributing factors. A SHA-256 request fingerprint protects idempotency keys against payload changes. The decision and an outbox event are committed in the same PostgreSQL transaction. A relay publishes pending records to Kafka with at-least-once delivery. Unit and integration tests, static analysis, CodeQL, and a CycloneDX SBOM provide reproducible engineering evidence.

Keywords: Microservices. Idempotency. Transactional outbox. Kafka. Fraud analysis.

SUMÁRIO

1 INTRODUÇÃO	5
2 PROCEDIMENTOS METODOLÓGICOS	6
3 FUNDAMENTAÇÃO TÉCNICA	7
4 ARQUITETURA DA SOLUÇÃO	8
5 MOTOR DE RISCO E EXPLICABILIDADE	9
6 IDEMPOTÊNCIA E CONCORRÊNCIA	10
7 OUTBOX TRANSACIONAL E KAFKA	11
8 SEGURANÇA E OBSERVABILIDADE	12
9 TESTES E AUTOMAÇÃO DA QUALIDADE	13
10 RESULTADOS	14
11 LIMITAÇÕES E TRADE-OFFS	15
12 CONCLUSÃO	16
REFERÊNCIAS	17

1 INTRODUÇÃO

Microserviços falham em pontos pouco visíveis durante uma demonstração feliz. Um cliente pode repetir a chamada depois de perder a resposta; o banco pode confirmar enquanto o broker está indisponível; duas instâncias podem processar a mesma chave ao mesmo tempo. O Sentinel Risk Service foi criado para estudar essas situações em um domínio de análise antifraude.

O problema investigado é como produzir uma decisão explicável, persistir o resultado sem duplicidade e disponibilizar um evento sem recorrer a transação distribuída. A solução utiliza Java 21, Spring Boot 4.1, PostgreSQL, Flyway e Kafka. O motor usa regras fixas para que o comportamento seja auditável; precisão estatística não faz parte do recorte.

1.1 Objetivo geral

Desenvolver e documentar um microserviço que analise transações, explique sua pontuação, trate repetição de requisições e publique decisões por meio de outbox transacional.

1.2 Objetivos específicos

Os objetivos específicos são isolar regras de domínio; validar o contrato HTTP; associar chave de idempotência a um fingerprint; gravar análise e evento atômicamente; explicitar a entrega pelo menos uma vez; autenticar chamadas; expor métricas; automatizar testes, análise estática e segurança; e registrar os limites que exigiriam outro desenho em produção.

2 PROCEDIMENTOS METODOLÓGICOS

O desenvolvimento foi incremental e guiado pelos riscos do fluxo. O primeiro incremento criou objetos de entrada, fatores, decisões e cinco regras. O segundo adicionou casos de uso e portas. Persistência, API, segurança e relay entraram depois que o motor podia ser exercitado sem Spring. Cada grupo coerente de mudanças recebeu um commit próprio.

A verificação combinou testes unitários e integração HTTP com H2 em modo PostgreSQL. O comando mvnw clean verify executa JUnit, JaCoCo, Checkstyle, SpotBugs e CycloneDX. O pipeline repete o build em Java 21, cria a imagem Docker e executa CodeQL. O Compose foi validado sintaticamente; o build local da imagem depende de um daemon Docker ativo.

Incremento	Risco observado	Evidência
Domínio	pontuação opaca	regras isoladas e fatores
Aplicação	retry duplicado	fingerprint e casos de uso
Persistência	dual write	migration e outbox
Operação	falha silenciosa	métricas, health e CodeQL

Quadro 1 - Desenvolvimento orientado aos riscos do fluxo

Fonte: elaboração própria (2026).

O README, os ADRs e este relatório passaram por revisão de linguagem para remover frases promocionais, repetições e conclusões vagas. A revisão não altera evidências nem transforma limitações em promessas.

3 FUNDAMENTAÇÃO TÉCNICA

3.1 Microserviço e fronteira

Um microserviço deve possuir um limite que faça sentido para o negócio e possa evoluir sem compartilhar seu modelo interno (NEWMAN, 2021). No Sentinel, a unidade é a decisão de risco. A API não administra clientes, cartões ou pagamentos; ela recebe identificadores opacos e publica o resultado.

3.2 Idempotência

Idempotência permite repetir uma operação sem multiplicar seu efeito. A chave sozinha não prova que o conteúdo é o mesmo. Por isso o serviço gera SHA-256 de uma representação canônica do comando e compara o fingerprint antes de reproduzir a resposta. Uma restrição única continua necessária diante de concorrência.

3.3 Outbox transacional

O padrão transactional outbox grava a alteração do negócio e uma mensagem na mesma transação local. Um processo separado encaminha o registro ao broker (RICHARDSON, 2018). O padrão remove o dual write do caminho da requisição, mas pode publicar duplicatas. O contrato precisa admitir e tratar esse efeito.

3.4 Explicabilidade

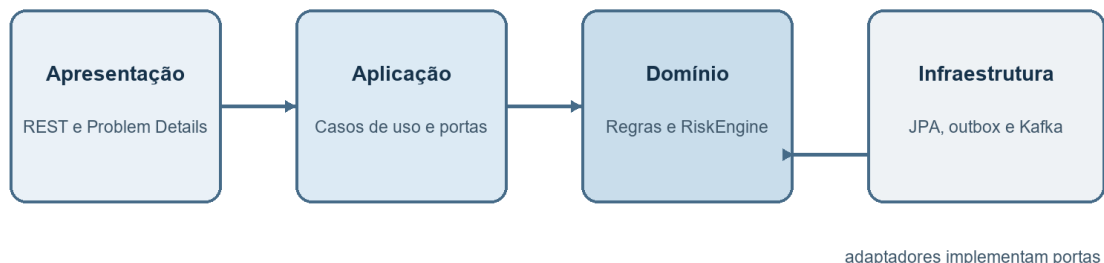
Cada regra devolve código, pontos e explicação. O score final é limitado a cem, e os fatores aparecem em ordem decrescente de peso. A decisão pode ser auditada sem inspecionar logs ou inferir o comportamento de um modelo estatístico.

4 ARQUITETURA DA SOLUÇÃO

A arquitetura organiza dependências em quatro áreas. O domínio contém modelos, regras e RiskEngine, sem imports do Spring. A aplicação coordena idempotência, consulta de velocidade, persistência e outbox. A infraestrutura implementa portas com JPA e Kafka. A apresentação converte HTTP em comandos e respostas.

Figura 1 - Limites e direção das dependências

Portas mantêm o domínio distante de HTTP, JPA e Kafka



Dependências do negócio apontam para dentro.

Fonte: elaboração própria (2026).

A separação deixa o motor testável sem contexto de aplicação e impede controllers de acessarem repositories. O caso de uso possui a transação que reúne análise e outbox. Configurações compõem objetos e propriedades, mas não carregam regras de pontuação.

O custo é um número maior de classes e mapeamentos. Ele é aceito porque substituição de adaptadores, testes rápidos e clareza dos limites são objetivos explícitos. Criar interfaces onde não há fronteira de substituição continuaria sendo evitado.

5 MOTOR DE RISCO E EXPLICABILIDADE

RiskEngine recebe um contexto com comando e quantidade de transações do cliente na última hora. Cada regra pode devolver um fator. O motor soma os pontos, limita o score a cem e escolhe APPROVED até 39, REVIEW entre 40 e 69 e DECLINED a partir de 70.

Fator	Condição	Pontos
ELEVATED_AMOUNT	valor a partir de 5.000	20
HIGH_AMOUNT	valor a partir de 10.000	35
COUNTRY_MISMATCH	país da compra difere do cartão	25
HIGH_RISK_MERCHANT	MCC 6051 ou 7995	30
UNUSUAL_HOUR	00:00 a 06:00 UTC	15
HIGH_VELOCITY	três transações recentes	30

Quadro 2 - Regras e pesos da prova de conceito

Fonte: elaboração própria (2026).

Os pesos não vieram de dados históricos. Eles existem para testar composição, limiares e explicação. Uma versão real precisaria calibrar políticas, medir falsos positivos, versionar regras e investigar vieses. Autorizar dinheiro com estes valores seria tecnicamente e eticamente inadequado.

O horário é avaliado em UTC para evitar comportamento diferente entre servidores. Valores monetários usam BigDecimal com duas casas. Códigos de moeda, país e MCC são normalizados antes de alcançar o motor.

6 IDEMPOTÊNCIA E CONCORRÊNCIA

O cabeçalho Idempotency-Key é obrigatório e aceita até cem caracteres. RequestFingerprint concatena campos normalizados em ordem estável e calcula SHA-256. Quando a chave já existe, fingerprint igual reproduz a decisão; fingerprint diferente retorna 409 Conflict.

Situação	Resposta	Efeito
chave nova	201 Created	grava análise e evento
mesma chave e corpo	201 Created	reproduz analysisId
mesma chave e outro corpo	409 Conflict	nenhuma nova análise
corrida entre instâncias	um insert vence	restrição única decide

Quadro 3 - Semântica da idempotência

Fonte: elaboração própria (2026).

Duas requisições concorrentes ainda podem calcular o risco antes da inserção. A restrição única impede dois registros; a perdedora recebe conflito e pode repetir para encontrar a vencedora. Reservar a chave antes do cálculo permitiria estados PROCESSING e COMPLETED, mas adicionaria tabela, expiração e recuperação.

A representação canônica faz parte do contrato interno. Incluir um campo novo ou mudar normalização pode alterar fingerprints. Esse ponto deve acompanhar a evolução de versão da API.

7 OUTBOX TRANSACIONAL E KAFKA

AnalyzeTransactionService salva RiskAnalysisEntity e OutboxEventEntity sob a mesma transação. A resposta HTTP não depende da disponibilidade do Kafka. OutboxRelay busca até cinquenta registros pendentes, e o serviço de publicação aguarda a confirmação do produtor antes de marcar PUBLISHED.

Figura 2 - Fluxo de persistência e publicação

A requisição confirma o banco antes de depender do broker



Fonte: elaboração própria (2026).

Uma falha mantém o evento pendente, incrementa attempts e registra mensagem limitada a quinhentos caracteres. Se o processo cair depois do ack e antes do commit, o evento pode reaparecer. O consumidor deve deduplicar por eventId; essa é a consequência explícita da entrega pelo menos uma vez.

Múltiplos relays exigiriam claim com FOR UPDATE SKIP LOCKED ou mecanismo equivalente. O desenho atual não promete coordenação eficiente entre várias instâncias, embora duplicatas continuem válidas pelo contrato.

8 SEGURANÇA E OBSERVABILIDADE

A API e as métricas exigem X-API-Key; o health check permanece público. A comparação da chave usa MessageDigest.isEqual. A configuração vem de variável de ambiente e não deve usar o valor local em rede compartilhada. Em produção, OAuth 2.1 com client credentials ou mTLS permitiria identidade e rotação.

X-Correlation-ID aceita somente letras, números, ponto, sublinhado e hífen, com limite de 64 caracteres. Valores fora do padrão são substituídos por UUID. O identificador aparece no MDC e na resposta, sem expor payload ou segredo nos logs.

Ameaça	Controle	Lacuna
retry duplicado	idempotência + fingerprint	corrida pode exigir retry
dual write	outbox transacional	entrega pode duplicar
segredo roubado	chave externa	sem identidade por cliente
evento preso	métrica de falha	sem dead-letter automática
log forging	correlation ID seguro	proxy também requer política

Quadro 4 - Ameaças selecionadas e controles

Fonte: threat model do projeto (2026).

Actuator oferece health probes e endpoint Prometheus. Os contadores sentinel.outbox.published e sentinel.outbox.failed distinguem confirmação e tentativa que permaneceu pendente. Um ambiente real ainda precisa alertar sobre idade do registro mais antigo e crescimento da fila.

9 TESTES E AUTOMAÇÃO DA QUALIDADE

A suíte separa regras, casos de uso, outbox e fluxo HTTP. Testes do domínio verificam normalização, pesos, limiares e score máximo. Testes da aplicação exercitam replay, conflito, tempo futuro e limite da chave. O outbox é testado nos caminhos de ack, falha e ausência do evento.

A integração inicia o contexto Spring, aplica a migration Flyway em H2 no modo PostgreSQL e usa MockMvc com a cadeia de segurança. Ela cobre acesso sem chave, health público, criação, leitura, replay, conflito, validação e recurso inexistente.

Controle	Finalidade	Regra
JUnit	comportamento	nenhuma falha
JaCoCo	regressão de cobertura	mínimo 85% de linhas
Checkstyle	consistência estática	zero violação
SpotBugs	padrões de defeito	threshold Low
CodeQL	análise de segurança	push, PR e agenda
CycloneDX	inventário de componentes	SBOM no verify

Quadro 5 - Controles automáticos de qualidade

Fonte: configuração Maven e GitHub Actions (2026).

Cobertura não substitui a análise dos cenários. O limite atua como alarme, enquanto testes direcionados protegem decisões que podem ter pouco peso percentual, como a janela de cinco minutos para relógio e a falha entre broker e banco.

10 RESULTADOS

A versão avaliada recebe transações, produz decisões explicáveis, consulta análises por UUID e reproduz a resposta associada à chave. O banco valida schema criado por Flyway. A mesma transação persiste fatores e evento. O relay publica JSON no tópico configurado e registra ack ou falha.

Critério	Resultado observado
Testes	32 aprovados antes da validação final
Cobertura	gate de 85% atendido
Checkstyle	zero violações
API	autenticação, validação e Problem Details
Persistência	Flyway + JPA validate
Eventos	outbox e Kafka com retry
Empacotamento	JAR, Dockerfile e Compose válido

Quadro 6 - Síntese da verificação

Fonte: build local executado em 16 ago. 2026.

Os testes localizaram duas diferenças relevantes do Spring Boot 4.1: Flyway e Kafka exigem starters modulares para ativar a autoconfiguração. A correção ocorreu antes da publicação. O episódio confirma a utilidade do teste que carrega o contexto completo em vez de limitar a suíte a mocks.

O histórico separa fundação, domínio, aplicação, persistência, API, outbox, testes, automação e documentos. README, OpenAPI, quatro ADRs, threat model e textos para LinkedIn acompanham o código.

11 LIMITAÇÕES E TRADE-OFFS

As regras usam pesos fixos e não representam um modelo antifraude calibrado. Não há avaliação de viés, monitoramento de falsos positivos, versionamento de política ou explicação para um modelo estatístico. O serviço evita dados de cartão e recebe somente identificadores opacos.

H2 acelera a integração, mas não reproduz todas as características do PostgreSQL. Testcontainers com banco e Kafka reais seria apropriado em homologação. O daemon Docker não estava ativo durante a verificação local, portanto o Compose foi validado sintaticamente e o pipeline ficou responsável pelo build da imagem.

O relay mantém a transação aberta enquanto aguarda o broker. Em volume alto, um claim curto seguido por publicação e confirmação separadas reduziria locks. Múltiplas instâncias também pediriam SKIP LOCKED. Uma dead-letter operacional seria necessária após tentativas persistentes.

API key reduz o escopo de demonstração, mas não oferece identidade, escopo ou rotação. Rate limit deve ficar no gateway. Retenção, criptografia em repouso, gestão de segredos e resposta a incidentes pertencem à plataforma e precisam ser definidos antes de aceitar tráfego real.

11.1 Evolução recomendada

Os próximos passos são testes com infraestrutura real, versionamento de políticas, claim concorrente do outbox, alerta por idade dos eventos, OAuth 2.1, dashboard Grafana e um consumidor que demonstre deduplicação.

12 CONCLUSÃO

O Sentinel Risk Service cumpriu o objetivo de demonstrar decisões de engenharia que aparecem quando uma API deixa o caminho feliz. A chave de idempotência associada ao fingerprint trata retries sem confundir corpos diferentes. O outbox remove o dual write da requisição. O contrato admite duplicatas e fornece eventId ao consumidor, em vez de prometer uma garantia que o desenho não sustenta.

A separação entre domínio, aplicação, apresentação e infraestrutura tornou as regras testáveis sem framework e manteve transação, JPA e Kafka nas bordas. A API acrescentou validação, Problem Details, autenticação simples e correlação de logs. Métricas e health probes permitem observar o relay.

As evidências incluem testes de comportamento e integração, cobertura mínima, análise estática, CodeQL, SBOM, migration, contrato OpenAPI e decisões registradas. O relatório também preserva limites: regras não calibradas, H2 na suíte, API key e relay sem claim distribuído.

O resultado é uma base para discutir consistência, entrega de eventos e segurança em entrevista técnica. O valor do projeto não está na quantidade de endpoints, mas na possibilidade de verificar por que cada escolha foi feita e em que contexto ela deixaria de ser suficiente.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR 14724: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2024.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR 10520: informação e documentação: citações em documentos: apresentação. Rio de Janeiro: ABNT, 2023.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR 6023: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2025.

FOWLER, Martin. Patterns of enterprise application architecture. Boston: Addison-Wesley, 2002.

GITHUB. CodeQL documentation. Disponível em: <https://docs.github.com/en/code-security/code-scanning>. Acesso em: 16 ago. 2026.

NEWMAN, Sam. Building microservices: designing fine-grained systems. 2. ed. Sebastopol: O'Reilly, 2021.

RICHARDSON, Chris. Microservices patterns. Shelter Island: Manning, 2018.

SPRING. Spring Boot 4.1.0 reference documentation. Disponível em: <https://docs.spring.io/spring-boot/>. Acesso em: 16 ago. 2026.

THE APACHE SOFTWARE FOUNDATION. Apache Kafka 4.3.1. Disponível em: <https://kafka.apache.org/community/downloads/>. Acesso em: 16 ago. 2026.

OWASP FOUNDATION. OWASP API Security Top 10. Disponível em: <https://owasp.org/API-Security/>. Acesso em: 16 ago. 2026.